



C Piscine

C 13

Preâmbulo: Este documento é o subject para o módulo C 13 da Piscine C na 42.

Versão: 6.0

Sumário

I	Instruções	2
II	Instruções de IA	4
III	Prefácio	7
IV	Exercício 00 : btree_create_node	8
V	Exercício 01 : btree_apply_prefix	9
VI	Exercício 02 : btree_apply_infix	10
VII	Exercício 03 : btree_apply_suffix	11
VIII	Exercício 04 : btree_insert_data	12
IX	Exercício 05 : btree_search_item	13
X	Exercício 06 : btree_level_count	14
XI	Exercício 07 : btree_apply_by_level	15
XII	Entrega e avaliação por pares	16

Capítulo I

Instruções

- Somente esta página serve como sua referência, não confie em rumores.
- Cuidado! Este documento pode mudar antes da submissão.
- Certifique-se de que você tenha as permissões apropriadas em seus arquivos e diretórios.
- Você deve seguir os **procedimentos de submissão** para todos os seus exercícios.
- Seus exercícios serão verificados e corrigidos por seus colegas de classe.
- Além disso, seus exercícios serão avaliados por um programa chamado **Moulinette**.
- **Moulinette** é meticulosa e estrita em sua avaliação. Ela é totalmente automatizada, e não há como negociar com ela. Para evitar surpresas desagradáveis, seja o mais completo possível.
- **Moulinette** não é tolerante. Se seu código não aderir à Norma, ela não tentará entendê-lo. **Moulinette** utiliza um programa chamado **norminette** para verificar se seus arquivos estão em conformidade com a Norma. TL;DR: Submeter trabalho que não passa na verificação da **norminette** não faz sentido.
- Esses exercícios são organizados por ordem de dificuldade, do mais fácil para o mais difícil. Nós **não** consideraremos um exercício mais difícil concluído com sucesso se um exercício mais fácil não estiver totalmente funcional.
- Usar uma função proibida é considerado trapaça. Trapaceiros recebem uma nota de **-42**, o que é inegociável.
- Você só precisa submeter uma função **main()** se pedirmos especificamente um **programa**.
- **Moulinette** compila com as seguintes flags: **-Wall -Wextra -Werror**, usando **cc**.
- Se seu programa não compilar, você receberá uma nota **0**.

- Você **não pode** deixar **nenhum** arquivo adicional em seu diretório além daqueles especificados na tarefa.
- Tem uma pergunta? Pergunte ao colega à sua direita. Se não, tente o colega à sua esquerda.
- Seu guia de referência se chama **Google / man / Internet / ...**
- Verifique a seção "Piscine C" do fórum na intranet ou na Piscine no Slack.
- Examine cuidadosamente os exemplos. Eles podem conter detalhes cruciais que não são explicitamente declarados na tarefa...
- Por Odin, por Thor! Use seu cérebro!!!
- Para os seguintes exercícios, usaremos a seguinte estrutura:

```
typedef struct      s_btree
{
    struct s_btree  *left;
    struct s_btree  *right;
    void            *item;
}                  t_btree;
```

- Você terá que incluir esta estrutura em um arquivo `ft_btree.h` e enviá-lo para cada exercício.
- A partir do exercício 01 em diante, usaremos nossa `btree_create_node`, então faça os arranjos necessários (poderia ser útil ter seu protótipo em um arquivo `ft_btree.h...`).

Capítulo II

Instruções de IA

● Contexto

A C Piscine é intensa. É seu primeiro grande desafio na 42 — um mergulho profundo na resolução de problemas, autonomia e comunidade.

Durante essa fase, seu objetivo principal é construir sua base — através do esforço, da repetição e, especialmente, da troca de **aprendizagem entre pares**.

Na era da IA, atalhos são fáceis de encontrar. No entanto, é importante considerar se o uso da IA está realmente ajudando você a crescer — ou simplesmente atrapalhando o desenvolvimento de habilidades reais.

A Piscine também é uma experiência humana — e, por enquanto, nada pode substituí-la. Nem mesmo a IA.

Para uma visão mais completa de nossa posição sobre a IA — como ferramenta de aprendizagem, como parte do currículo de TIC e como uma expectativa crescente no mercado de trabalho — consulte as perguntas frequentes disponíveis na intranet.

● Mensagem principal

- ✎ Construa bases sólidas sem atalhos.
- ✎ Desenvolva realmente habilidades técnicas e de poder.
- ✎ Experimente a verdadeira aprendizagem entre pares, comece a aprender como aprender e resolver novos problemas.
- ✎ A jornada de aprendizagem é mais importante que o resultado.
- ✎ Aprenda sobre os riscos associados à IA e desenvolva práticas de controle eficazes e contramedidas para evitar armadilhas comuns.

● Regras para o aluno:

- Você deve aplicar o raciocínio às tarefas atribuídas, especialmente antes de recorrer à IA.
- Você não deve pedir respostas diretas à IA.
- Você deve aprender sobre a abordagem global da 42 em relação à IA.

● Resultados da fase:

Dentro desta fase fundamental, você obterá os seguintes resultados:

- Obter bases adequadas em tecnologia e codificação.
- Saber por que e como a IA pode ser perigosa durante esta fase.

● Comentários e exemplo:

- Sim, sabemos que a IA existe — e sim, ela pode resolver seus projetos. Mas você está aqui para aprender, não para provar que a IA aprendeu. Não perca seu tempo (nem o nosso) apenas para demonstrar que a IA pode resolver o problema proposto.
- Aprender na 42 não é sobre saber a resposta — é sobre desenvolver a capacidade de encontrá-la. A IA lhe dá a resposta diretamente, mas isso impede você de construir seu próprio raciocínio. E o raciocínio leva tempo, esforço e envolve falhas. O caminho para o sucesso não deve ser fácil.
- Lembre-se de que durante os exames, a IA não está disponível — sem internet, sem smartphones, etc. Você perceberá rapidamente se confiou muito na IA em seu processo de aprendizagem.
- A aprendizagem entre pares o expõe a diferentes ideias e abordagens, melhorando suas habilidades interpessoais e sua capacidade de pensar de forma divergente. Isso é muito mais valioso do que apenas conversar com um bot. Portanto, não seja tímido — converse, faça perguntas e aprenda juntos!
- Sim, a IA fará parte do currículo — tanto como ferramenta de aprendizagem quanto como tópico em si. Você terá até a chance de construir seu próprio software de IA. Para saber mais sobre nossa abordagem crescente, consulte a documentação disponível na intranet.

✓ Boa prática:

Estou travado em um novo conceito. Pergunto a alguém próximo como ele abordou isso. Conversamos por 10 minutos — e de repente, clica. Entendi.

✗ Má prática:

Usei secretamente a IA, copiei um código que parece certo. Durante a avaliação entre pares, não consigo explicar nada. Eu falho. Durante o exame — sem IA — estou travado novamente. Eu falho.

Capítulo III

Prefácio

Aqui está a lista de lançamentos de **Venom** :

- In League with Satan (single, 1980)
- Welcome to Hell (1981)
- Black Metal (1982)
- Bloodlust (single, 1983)
- Die Hard (single, 1983)
- Warhead (single, 1984)
- At War with Satan (1984)
- Hell at Hammersmith (EP, 1985)
- American Assault (EP, 1985)
- Canadian Assault (EP, 1985)
- French Assault (EP, 1985)
- Japanese Assault (EP, 1985)
- Scandinavian Assault (EP, 1985)
- Manitou (single, 1985)
- Nightmare (single, 1985)
- Possessed (1985)
- German Assault (EP, 1987)
- Calm Before the Storm (1987)
- Prime Evil (1989)
- Tear Your Soul Apart (EP, 1990)
- Temples of Ice (1991)
- The Waste Lands (1992)
- Venom '96 (EP, 1996)
- Cast in Stone (1997)
- Resurrection (2000)
- Anti Christ (single, 2006)
- Metal Black (2006)
- Hell (2008)
- Fallen Angels (2011)

O tema de hoje parecerá mais fácil se você ouvir **Venom**.

Capítulo IV

Exercício 00 : btree_create_node

	Exercício : 00
	btree_create_node
	Pasta de entrega : <i>ex00/</i>
	Arquivos para entregar : btree_create_node.c , ft_btree.h
	Funções ou bibliotecas autorizadas : malloc

- Crie a função **btree_create_node** que aloca um novo elemento. Ela deve inicializar seu **item** com o valor do argumento, e todos os outros elementos com 0.
- O endereço do nó criado é retornado.
- Aqui está como ela deve ser prototipada:

```
t_btree *btree_create_node(void *item);
```

Capítulo V

Exercício 01 : btree_apply_prefix

	Exercício : 01
	btree_apply_prefix
	Pasta de entrega : <i>ex01/</i>
	Arquivos para entregar : <code>btree_apply_prefix.c</code> , <code>ft_btree.h</code>
	Funções ou bibliotecas autorizadas : Nenhuma

- Crie uma função `btree_apply_prefix` que aplica a função dada como argumento ao `item` de cada nó, usando a `travessia prefixada` para percorrer a árvore.
- Aqui está como ela deve ser prototipada:

```
void btree_apply_prefix(t_btree *root, void (*applyf)(void *));
```

Capítulo VI

Exercício 02 : btree_apply_infix

	Exercício : 02
	btree_apply_infix
	Pasta de entrega : <i>ex02/</i>
	Arquivos para entregar : <code>btree_apply_infix.c</code> , <code>ft_btree.h</code>
	Funções ou bibliotecas autorizadas : Nenhuma

- Crie uma função `btree_apply_infix` que aplica a função dada como argumento ao `item` de cada nó, usando a `travessia infixa` para percorrer a árvore.
- Aqui está como ela deve ser prototipada:

```
void btree_apply_infix(t_btree *root, void (*applyf)(void *));
```

Capítulo VII

Exercício 03 : btree_apply_suffix

	Exercício : 03
	btree_apply_suffix
	Pasta de entrega : <i>ex03/</i>
	Arquivos para entregar : <code>btree_apply_suffix.c</code> , <code>ft_btree.h</code>
	Funções ou bibliotecas autorizadas : Nenhuma

- Crie uma função `btree_apply_suffix` que aplica a função dada como argumento ao `item` de cada nó, usando a `travessia sufixada` para percorrer a árvore.
- Aqui está como ela deve ser prototipada:

```
void btree_apply_suffix(t_btree *root, void (*applyf)(void *));
```

Capítulo VIII

Exercício 04 : btree_insert_data

	Exercício : 04
	btree_insert_data
	Pasta de entrega : <i>ex04/</i>
	Arquivos para entregar : btree_insert_data.c , ft_btree.h
	Funções ou bibliotecas autorizadas : btree_create_node

- Crie uma função **btree_insert_data** que insere o elemento **item** em uma árvore. A árvore passada como argumento será ordenada: para cada nó, todos os elementos menores são localizados no lado esquerdo, e todos os elementos maiores ou iguais são localizados no lado direito. Também passaremos uma função de comparação similar a **strcmp** como argumento.
- O parâmetro **root** aponta para o nó raiz da árvore. Quando chamada pela primeira vez, ele deve apontar para **NULL**.
- Aqui está como ela deve ser prototipada:

```
void btree_insert_data(t_btree **root, void *item, int (*cmpf)(void *, void *));
```

Capítulo IX

Exercício 05 : btree_search_item

	Exercício : 05
	btree_search_item
	Pasta de entrega : <i>ex05/</i>
	Arquivos para entregar : btree_search_item.c , ft_btree.h
	Funções ou bibliotecas autorizadas : Nenhuma

- Crie uma função **btree_search_item** que retorna o primeiro elemento relacionado aos dados de referência dados como argumento. A árvore deve ser percorrida usando a **travessia infixa**. Se o elemento não for encontrado, a função deve retornar **NULL**.
- Aqui está como ela deve ser prototipada:

```
void *btree_search_item(t_btree *root, void *data_ref, int (*cmpf)(void *, void *));
```

Capítulo X

Exercício 06 : btree_level_count

	Exercício : 06
	btree_level_count
	Pasta de entrega : <i>ex06/</i>
	Arquivos para entregar : <code>btree_level_count.c</code> , <code>ft_btree.h</code>
	Funções ou bibliotecas autorizadas : Nenhuma

- Crie uma função `btree_level_count` que retorna o tamanho do maior ramo passado como argumento.
- Aqui está como ela deve ser prototipada:

```
int btree_level_count(t_btree *root);
```

Capítulo XI

Exercício 07 : btree_apply_by_level

	Exercício : 07
btree_apply_by_level	
Pasta de entrega : <i>ex07/</i>	
Arquivos para entregar : btree_apply_by_level.c , ft_btree.h	
Funções ou bibliotecas autorizadas : malloc , free	

- Crie uma função **btree_apply_by_level** que aplica a função passada como argumento a cada nó da árvore. A árvore deve ser percorrida nível por nível. A função chamada receberá três argumentos:
 - O primeiro argumento, do tipo **void ***, corresponde ao item do nó;
 - O segundo argumento, do tipo **int**, corresponde ao nível em que o encontramos: 0 para a raiz, 1 para os filhos, 2 para os netos, etc.;
 - O terceiro argumento, do tipo **int**, vale 1 se for o primeiro nó do nível, ou 0 caso contrário.
- Aqui está como ela deve ser prototipada:

```
void btree_apply_by_level(t_btree *root, void (*applyf)(void *item, int current_level, int is_first_child));
```


Capítulo XII

Entrega e avaliação por pares

Envie sua tarefa para o seu repositório `Git` como de costume. Apenas o trabalho dentro do seu repositório será avaliado durante a defesa. Certifique-se de verificar novamente os nomes dos arquivos para garantir que estão corretos.



Você deve enviar apenas os arquivos explicitamente exigidos pelas instruções do projeto.